Ultralearning

The following is the notes I took years ago on the book `Ultralearning` by Scott Young. The bombastic title and promise to learn virtually anything quickly makes it sound as if its the typical marketing-powered fluff-filled nonfiction book stores are overflowing with, but something about this book stuck with me. After finishing it I quickly went back and wrote these thoughts down. While I don't follow his layout of plans regularly, I have used it to guide a lot of my own self education.

I've successfully utilized it when I needed to refresh on mathematics for my Master's degree (a host of skills that atrophied sigificantly for the dozen years between undergrad and the masters). I've also used it to self-study subjects like Robotics and Deep Learning (though I did decide in the end to go for the Master's accreditation).

I share it here with hopes that someone finds it useful.

Principle 1 - Metalearning

Determine What, Why and How

What - What are you learning? Self explanatory for the high level, but get specific. Not "I will learn chinese" but "I will learn to speak chinese for a 15 minute conversation." Clear concise goals, that you can then break down into further goals

Why - why do you want to know it? If you want to know XYZ in order to do ABC, then it orients your goals differently then merely doing a deep dive on XYZ. If there is a why controlling your need to learn this project, that will shape your approach to learning it. It also checks to see if you even should continue with your desired subject.

- Talk to experts - they will tell you if you're focussing on the wrong information. Case in point - if you want to be a web programmer, data structures aren't strictly necessary to get started.

How - What is your approach going to be? How much time is appropriated for your tasks? What methodologies will you use?

- Break it down into Concepts, Facts, and Procedures.
  - **Concepts:** Core concepts and ideas you're going to need to understand in order to get anywhere with this subject. IE deep learning requires understanding back propagation, likely linear algebra.
  - **Facts:** What needs to be memorized. You won't necessarily know this at first. If learning a language, this would be vocab or grammar. If calculus, it's formulas or how derivatives work or the trignometric identities.
  - **Procedures:** Anything that needs to be practiced
- Benchmarking and Emphasize / Exclude Method:

    - **Benchmarking:** Find other ways people commonly learning the skill or topic. This gives you a start on what to focus on.
        - IE Curriculum for degrees on the topic
        - Existing classes
    - **Emphasize/Exclude:** Modify what you discover in benchmarking, cutting down to only what you need to know. Focus on parts of the topic that align with your goals from WHY. Omit anything that would delay you and isn't heavily needed.
- **Plan** - spend about 10 percent for shorter projects and 5 percenter for larger projects planning and preparing before starting. Research has diminishing returns, yes, but is still valuable.

Principle 2 - Focus

Identify the problems distracting you from your project.

Procastination

- Identify when you're procastinating, ask yourself how you go to that. Focus on habit building here.
- Pompooro Technique
- If work is too hard to work on, then try and make it easier, less intense. But don't make it too easy, or you'll get bored because of the ease.

If you're getting distracted and can't maintain distraction, eliminate distractions. Get into a "flow state" however you can. Schedule out time that you can focus for studies and work.

- Environment is the first source of distraction. Make sure you're in a spot you can work. Don't use excuses to introduce simple distractions, like music or leaving the TV on.
- If the task is harder, it may require more time to warm up. If you feel stuck, take a ten minute break and come back
- Your mind - be cognizate of negative emotions and troublesome worries that are affecting your performance.
- Failing to create the right kind of focus - for memorization, high intensity focus is great, but for creative tasks, relaxed focus to allow a little bit of mind wandering is better.

Principle 3 - Directness

Transfer learning is difficult - historically education has trouble teaching core concepts and getting students to demonstrate "transfering" these core concepts to actual practical outcomes. You get around this by focusing on directly what you wish to learn. IE if learning a language, speak it. If learning math, do the problems. If learning programming, program!

Project-Based Learning

Focus around producing projects (or one major project) that forces you to practice learned skills immediately. It also highlights what you can focus on and what you can exclude from your education.

Immersive Learning

Surround yourself with a target environment where the skill is practiced. IE a country that speaks the language, or surround yourself with other coders, etc.

Flight Simulator Method

Find simulacrons that allow you to repeatedly practice the skill in simulated circumstances - useful when direct practice is prohibited.

Overkill Approach

Complete, intensive immersion. IE no speaking english until you master a language, or doing only your project for days or weeks. It feels intense and uncomfortable but that means you're learning rapidly and pushing yourself.

Principle 4 - Drill

Find your "rate-limiting" factor - what's holding you back?

Drill methods:

Direct-then-drill - Do the project head-on via direct (see last section) and then have moments of awkwardness or unfamiliarity turned into drill exercsies for you to focus on shortly after.

Time slicing

Divide longer sequences of the task into smaller isolated parts. IE learning an instrument - break down the music into smaller sections to practice. For computer programming, focus on smaller sub-modules and applications to focus on specific skills.

Cognitive Components

Combine linked aspects when drilling to build a mental connection

Copycat

Copy parts you don't want to drill. Copy drawings or photographs if you just want to practice your coloring techniques. For programming, skip the website or the API if you want to focus on one or the other.

Magnifying Glass Method

Focus on core components of the larger skillset itself, especially sections you have trouble on.

Prerequisite Chaining

Increasingly add prerequisites as to increase the difficulty of drills.

Principle 5 - Retrieval

Difficulty is desireable. If you feel challenged, you're more likely to remember what you're studying and working on. Aim to stay challenged and uncomfortable, but not overwhelmed.

Also, it seems that the best way to retrieve something is to attempt retrieving it immediately - even before there is anything left to retrieve. IE - try and test and retrieve it from memory even before you've actually learned anything.

Tactics for retrieval

Flash cards

Tried and true, great when memorization is a large part of what you're doing - IE vocab, formulas, derivatives, etc. There are plenty of programs such as Anki, which will handle mixing these up for you automatically and setting a "retrieval schedule" for you.

Free Recall

Immediately after reading a section of a book or sitting through a lecture, write down a summary of everything you can remember from the lecture. Afterwards, see what you missed - what details need review? This also helps prime it in your mind.

Question-Book Method

As you read or cover material, write down questions about the content. You can use this later as a study aid. You must avoid becoming lazy and merely writing down surface-level questions, however. For instance - writing a question about the date an algorithm was introduced, rather than a question about the core ideas behind it.

Self-Generated Challenges

If the information can directly be used to do something, do it. IE learning a programming language and finding a new function of the language - actively go and use the feature somewhere to test it out and build familiarity with the concept.

Closed-Book Learning

Create a core concept-map of what you've learned and covered, without looking at any notes or your books. Then see what aspects you've missed in your concept maps.

Principle 6. Feedback

Feedback can be a double edged sword. Constructive feedback can increase your retention and learning of a topic. Feedback that targets an ego or emotion can hamper learning a topic. Even positive feedback that targets an ego can be damaging (ie: "You're so smart!")

First, determine what kind of feedback you need. Do you need feedback from someone knowledgeable about the topic, or the feedback of someone else in general? IE language or performance-related learnings, respectively.

Types of Feedback

There are three types of feedback: outcome feedback, informational feedback, and corrective feedback.

Outcome Feedback

This is whether or not you did it correctly. Deep learning - did the model train. Language - did the person understand you. Performance - did people actually like you. Very binary. This is the broadest-scale feedback to get, and can be useful, but typically does not give insight as to where to improve.

Informational Feedback

What are you doing wrong? Unfortunately, usually not paired with how to fix what you're doing wrong unless the feedback is being given by someone whom has already mastered your target skill (where it would then be correctional feedback).

This type of feedback highlights more specifically where you're going wrong. IE a computer programmer gets some errors that, while mysterious, at least points towards where the problem might be.

Corrective Feedback

Feedback that directly tells you what's wrong and how to fix it. If you're studying a topic like math, find problem sets and tests with answer keys so you can see where you went awry.

Feedback should be immediate. Do not wait to grade your own answers, or to test what you're doing. Build/perform/act within your project and test it immediately to try and garner any feedback possible. This should then be considered, weighed, and you should adjust accordingly where possible.

How to improve your feedback:

Noise Cancellation

Isolate the signal from the noise. You'll get feedback sometimes on random factors that don't affect what you're trying to accomplish. Isolate and ignore this feedback - hone in on what actually matters relative to your goal. IE if you're writing blog posts, don't go on just raw views - do more specific analytics to determine if those that came actually read the blog post. This is far more telling as raw popularity doesn't necessarily link to writing quality at first.

Hitting the Difficulty Sweet Spot

You do not want to do tasks where the feedback is guarenteed good or bad - the difficulty sweet spot is key to ensuring you're actually learning and getting useful feedback from your experiences.

Metafeedback

This is where you're feedback is being applied directly to your strategy and prepared plans on how you're learning for your project. Set a rating that you can track as you're learning - IE chess - check your ELO over time. LSAT ratings as you do mock exams. Etc.

High-Intensity Rapid Feedback

Get more feedback, as often as possible. This is not unusual to ultra-learning. IE if learning a language, constantly talk to people despite not knowing the language.

Principle 7. Retention

Learn techniques to improve memory retention so you can actually keep what you learn.

Spacing: repeat to remember

The most obvious - repeat regularly. It's better to learn things over a several periods than a singular one. Look at spaced-repetition-systems (SRS) is a great tool for memorizing facts and such, but has limits on helping other skills.

Proceduralization - automatic will endure

If you manage to make the learned behaviors into automatic habits, they will stick around longer. Build up habits for automatic responses.

Overlearning: Practice Beyond Perfect

Reviewing material far beyond when you understand it will greatly increase retention of material. It also aids in the establishment of habits which are difficult to break.

Mnemomincs

Creating mental images when associated can help recall of key facts. This only works well with memorization of facts and concepts.

Principle 8 - Intuition

Transfer is one of the hardest aspects of education. By focusing on directness (Principle 3), you can avoid this sometimes, but there are times where you have to apply abstract concepts you're learning and drilling to real-world applications in order to reach your goals. In this case, developing an intuition about problems within your project-space becomes crucial. IE - don't memorize formulas for math and fail to realize when its applicable when a slightly different problem presents itself to you. Aim to learn how things can be bent to apply to various scopes that you'll encounter as you learn the skill.

How to build your intuition:

1. Don't give up on hard problems easily. Challenging problems will reward you, but also deplete your motivation towards the problem. Set "struggle timers" - if you feel like giving up on a tough problem, work through it for ten more minutes to push just a bit further. Even if you fail at this point, the additional time will help you better recall the problem if you do encounter a solution later.

2. Prove things to understand them. Don't take things at face value - work out why these things are true. This helps you understand things from a base level, creating a stronger understanding of more complicated interactions later. The example used here is the bicylce drawings - people think

they understand bikes, but when asked to draw one out of memory get the core structure of the chains, frame, and even pedal positions wrong!

3. Always start with a concrete example. Avoid the abstract if you can - human beings are bad at it. By developing a set example you'll be able to mentally imagine and work thorugh a problem better, as well as build your own recall.

4. Don't fool yourself. AKA avoid the Dunning-Kruger effect! To avoid this, ask lots of questions, even simple ones. You'll find that there's some simpler facts that allude you that you need to shore up your understanding of things - that's okay!

The "Feynman" Technique (probably not really his technique):

1. Write down the concept or problem you want to understand at the top of a piece of paper

2. In the space below, explain the idea how would convey the idea to someone whom has never heard it before. If it's a concept, ask yourself how you would convey the idea to someone whom has never heard of it before. If it's a problem, explain how to solve it and - crucially - why that solution procedure makes sense to you. Go down a bit deeper than you think is necessary to ensure you've shored up the basics!

3. When you get stuck, ie your understanding is lacking - go back to your books/notes/resources and find the answer to complete the exercise.

The crux of this is to avoid "explanatory depth" - ie you think you understand it as you can explain it, but really you're lacking key details that would exist in a better understanding.

When you apply this, you'll be in one of three possible applications:

1. **Application 1** - When you don't understand it at all. Do it slowly. It will be painful. Start breaking the problem down into smaller and smaller chunks. If reading a complex paper, break it down all the way to paragraphs and then individual sentences. Then start building back up. If it's a complex formula, break it down to the simplest arithmatic and work your way back up to building blocks you *do* understand.

2. **Application 2** - For problems you can't seem to solve. Avoid summarization!

3. **Application 3** - For expanding your intuition. If you feel truly confident about the base level, you can begin to skip the lower leve concepts and instead focus on generating illustrative examples, analogies, or visualizations that demonstrate the harder concepts.

Principle 9 - Experimentation

Experimentation is the key to mastery. Experimentation tests you on all levels, including the basics. this prevents the basics from stagnating and ensures better recall later. Skills also tend to reward not just proficiency but also orginiality and flexibility - experimentation builds both of these.

There are three types of experimentation:

1. **Experimenting with learning resources** - Try different ways to learn your chosen subject - differnt materials, methods of practice and drilling, etc. You need to find if you learn certain kinds of topics better in audio, video, or text form, for instance. So feel free to switch around how you're consuming your information throughout the process to hone in on what works for you.

2. **Experimenting with technique** - Ask yourself not only "How can I learn this?" but also "What can I learn next?" When you complete the first skill, pick adjacent skillsets that expand upon the horizons of your skill. Do mini projects and then, at the end, determine if you want to try going into a different direction or continue along your chosen path of specialization.

3. **Experimenting with style** - Try approaching the skill from different angles. If it's writing, try writing a different kind of story/blogpost/article, or change the entire style of the article. Try copying other writer's styles. If doing drawing, switch up the type of drawing to a different medium. Etc.

Experiment tactics:

1. **Copy, then create** - Copy those that went before you. Modify it and make it yourself. Re-create it in a unique way.

2. **Compare methods side-by-side** - Split your days studying or project work in half, applying a different approach for each. Compare the results side by side over time to determine what's working for you.

3. **Introduce new constraints** - Constraints are the key to creativity. Add constraints to force you to be creative and try new directions.

4. **Hybrid skills** - Combine skills you already have with your newly acquired skills to create a unique hybrid to you, and go down that path. Combining skills also help you establish a better baseline.

5. **Explore the extremes** - Purposely push yourself.

How to do your first ultralearning project

RESEARCH

1. What topic and WHY
    - Set a specific scope - not "I will learn this language" but "I will be able to hold a fifteen minute conversation with a native speaker on average topics without needing aid"
2. The primary resource you're going to use
    - List out what you found ahead of time. Acquire them before hand.
3. Benchmark how others have learned the skill before you.
4. Direct practice activities - projects to build, or drills you can run through related to your subject
5. Backup materials - material that you may cover, but not directly. Keep it on hand in case you get stuck or need to adjust your plan.

SCHEDULE YOUR TIME

Make a schedule and stick to it. Don't overdo it. Intensity is good, but don't commit to an unrealistic schedule. Track your progress so you know if you have to adjust your schedule. Make sure that you clear out time that you can focus. Decode on an appropriate length of time for your project - having a due date helps! If you're doing a longer project (6+ months) try a pilot week where you test out a schedule so you know whether or not you can maintain it realistically. Build in buffer time for unknowns.

EXECUTE

Follow the Principles outlined above

REVIEW THE RESULTS

Routinely review your progress. Is the schedule working? Are the materials working out? Are you approximately where you should be in understanding? Is your method of getting feedback sufficient? Is that feedback pointing towards good progress? Don't be